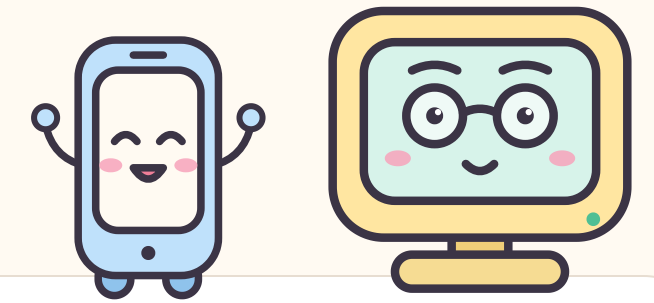




第 9 集 · 约 26 分钟

服务端②：存储

同一个改名函数，Python 不覆盖，Rust 覆盖



从零写出 LanShare

这一集你会学到

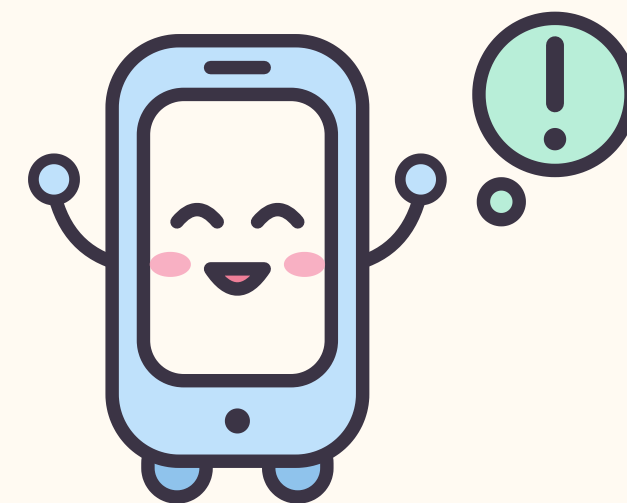
- 1. 共享文件夹的三道防线：名字检查、canonicalize、只认普通文件
- 2. 返回迭代器的函数：impl Iterator、move 闭包和生命周期 '_
- 3. fs::rename 在 Windows 上会覆盖；用 MoveFileExW 写一个不覆盖的改名
- 4. “先查再改”为什么有漏洞（TOCTOU），以及变异测试怎么揪出它



先想一个问题

把 v1 的 `os.rename(tmp, final)` 翻译成 Rust 的 `fs::rename(tmp, &final)`，在 Windows 上行为一样吗？

不一样。目标文件已经存在时，Python 拒绝改名；Rust 直接覆盖，还返回成功。





storage.rs 管四件事

● 列文件：只列共享文件夹里的普通文件

不列点开头的文件、上传中的临时文件、子目录、指向文件夹外面的链接

● 找文件：名字可疑就当“不存在”

下载时按名字定位，绝不能走出共享文件夹

● 收文件：先写临时文件，收齐再改名，重名加序号，绝不覆盖

.lanshare-上传号.part，开头的点让它在列表里隐身

● 打扫：启动时删掉上次崩溃留下的临时文件

10 分钟没动过的才删



开头：常量和两个结构体

storage.rs crates/lanshare/src/

第 7-26 行 • 共 26 行

```

7 use std::sync::Mutex;
8 use std::time::{Duration, SystemTime, UNIX_EPOCH};
9
10 pub const TEMP_PREFIX: &str = ".lanshare-";
11 pub const TEMP_SUFFIX: &str = ".part";
12 const STALE_TEMP: Duration = Duration::from_secs(600);
13
14 #[derive(Debug, Clone)]
15 pub struct FileEntry {
16     pub name: String,
17     pub size: u64,
18     /// Unix 秒
19     pub mtime: f64,
20 }
21
22 pub struct Storage {
23     root: PathBuf,
24     /// 选名 + 改名必须原子：两个同名上传同时完成时不能互相覆盖。
25     rename_lock: Mutex<()>,
26 }

```



候选文件名：生成器和迭代器

Python v1 • storage.py

```
1 def _candidate_names(name):
2     stem, ext = os.path.splitext(name)
3     yield name
4     n = 1
5     while True:
6         yield "%s (%d)%s" % (stem, n, ext)
7         n += 1
```

Rust v2 • storage.rs

```
1 fn candidate_names(name: &str) -> impl Iterator<Item = String> + '_ {
2     let (stem, ext) = match name.rfind('.') {
3         Some(i) if i > 0 => name.split_at(i),
4         _ => (name, ""),
5     };
6     std::iter::once(name.to_string())
7         .chain((1u32..).map(move |n| format!("{stem} ({n}){ext}")))
8 }
```



两个小工具

storage.rs crates/lanshare/src/

第 20-39 行 · 共 39 行

```

20  }
21
22  pub struct Storage {
23      root: PathBuf,
24      /// 选名 + 改名必须原子：两个同名上传同时完成时不能互相覆盖。
25      rename_lock: Mutex<()>,
26  }
27
28  /// `名字.ext`、`名字 (1).ext`、`名字 (2).ext`.....
29  fn candidate_names(name: &str) -> impl Iterator<Item = String> + '_ {
30      let (stem, ext) = match name.rfind('.') {
31          Some(i) if i > 0 => name.split_at(i),
32          _ => (name, ""),
33      };
34      std::iter::once(name.to_string()).chain((1u32..).map(move |n| format!("{stem} ({n}){ext}")))
35  }
36
37  fn unix_seconds(t: SystemTime) -> f64 {
38      t.duration_since(UNIX_EPOCH).map(|d| d.as_secs_f64()).unwrap_or(0.0)
39  }

```



创建：路径先变成绝对路径

● ● ●

storage.rs

crates/lanshare/src/

第 43-62 行 · 共 62 行

```

43     pub fn new(root: impl Into<PathBuf>) -> Self {
44         let root = root.into();
45         let root = std::path::absolute(&root).unwrap_or(root);
46         let _ = fs::create_dir_all(&root);
47         Storage { root, rename_lock: Mutex::new(()) }
48     }
49
50     pub fn root(&self) -> &Path {
51         &self.root
52     }
53
54     /// 运行中目录被人删了就重建。
55     pub fn ensure_root(&self) -> io::Result<()> {
56         fs::create_dir_all(&self.root)
57     }
58
59     fn canonical_root(&self) -> io::Result<PathBuf> {
60         self.ensure_root()?;
61         fs::canonicalize(&self.root)
62     }

```



列文件：一条迭代器流水线



storage.rs

crates/lanshare/src/

第 69–88 行 · 共 88 行

```
69         .filter_map(Result::ok)
70         .filter_map(|entry| {
71             let name = entry.file_name().into_string().ok()?;
72             if name.starts_with('.') {
73                 return None;
74             }
75             let real = fs::canonicalize(entry.path()).ok()?;
76             if real.parent()? != root {
77                 return None;
78             }
79             let meta = fs::metadata(&real).ok()?;
80             if !meta.is_file() {
81                 return None;
82             }
83             Some(FileEntry { name, size: meta.len(), mtime: meta.modified().map(unix_seconds).unwrap_or(0.0) })
84         })
85         .collect();
86     files.sort_by(|a, b| b.mtime.total_cmp(&a.mtime));
87     files
88 }
```



下载时，这些名字会怎样

请求的名字	想干什么	被哪一道拦下
../secret.txt	走出共享文件夹	第一道：以点开头、含斜杠
.lanshare-x.part	下载别人上传了一半的临时文件	第一道：以点开头
a.txt:stream	NTFS 的“备用数据流”，藏在文件里的另一份数据	第一道：含冒号
CON	Windows 保留的设备名	后两道：它不是共享文件夹里的普通文件
link.txt（指向外面的符号链接）	借链接读到外面的文件	第二道：canonicalize 后父目录不对
folder	下载一个子文件夹	第三道：不是普通文件



按名字找文件：三道防线

storage.rs crates/lanshare/src/

第 87-106 行 · 共 106 行

```
87     files
88 }
89
90 /// 共享目录里名为 `name` 的普通文件的真实路径；名字可疑、不存在、越界时返回 None。
91 pub fn resolve(&self, name: &str) -> Option<PathBuf> {
92     if name.is_empty() || name.starts_with('.') || name.contains(['/', '\\', ':', '\\0']) {
93         return None;
94     }
95     let root = self.canonical_root().ok()?;
96     let real = fs::canonicalize(self.root.join(name)).ok()?;
97     if real.parent()? != root || !fs::metadata(&real).ok()?.is_file() {
98         return None;
99     }
100     Some(real)
101 }
102
103 /// 某次上传的临时文件路径（隐藏，列表里看不到）。
104 pub fn temp_path(&self, upload_id: &str) -> PathBuf {
105     self.root.join(format!("{TEMP_PREFIX}{upload_id}{TEMP_SUFFIX}"))
106 }
```



目标文件已经存在时，改名会怎样（Windows）

调用	结果
Python <code>os.rename(a, b)</code>	拒绝：FileExistsError，错误码 183
Python <code>os.replace(a, b)</code>	覆盖 b
Rust <code>fs::rename(a, b)</code>	成功返回，b 被悄悄覆盖



第一版：先查 exists() 再改名

```
1 // 审查之前的 finalize (节选)
2 if target.exists() {
3     continue;
4 }
5 match fs::rename(temp, &target) {
6     Ok(()) => return Ok(candidate),
7     // 别的程序恰好同时建了同名文件: Windows 上 rename 不覆盖, 换下一个名字
8     Err(e) if e.kind() == io::ErrorKind::AlreadyExists => continue,
9     // .....
10 }
```



修法：两件事

● 不先查 `exists()`，直接尝试改名

被占了就换下一个名字，检查和改名合成一步

● 改名本身必须“不覆盖”

Windows 上调 `MoveFileExW`，不带“替换已存在文件”的标志

● 锁还留着：管住我们自己的并发上传

不覆盖的改名，管住别的程序



改名落盘：finalize

storage.rs crates/lanshare/src/

第 126–145 行 · 共 146 行

```
126
127     /// 删掉上次异常退出留下的临时文件（10 分钟没动过的），返回删除个数。
128     pub fn cleanup_stale_temp(&self) -> usize {
129         let Ok(entries) = fs::read_dir(&self.root) else { return 0 };
130         let now = SystemTime::now();
131         entries
132             .filter_map(Result::ok)
133             .filter(|e| {
134                 let name = e.file_name().to_string_lossy().to_string();
135                 name.starts_with(TEMP_PREFIX) && name.ends_with(TEMP_SUFFIX)
136             })
137             .filter(|e| {
138                 e.metadata()
139                     .and_then(|m| m.modified())
140                     .map(|t| now.duration_since(t).unwrap_or_default()) > STALE_TEMP)
141                     .unwrap_or(false)
142             })
143             .filter(|e| fs::remove_file(e.path()).is_ok())
144             .count()
145     }
```



rename_no_replace: 调用 MoveFileExW

storage.rs crates/lanshare/src/

第 161–180 行 • 共 180 行

```
161 pub fn rename_no_replace(from: &Path, to: &Path) -> io::Result<()> {
162     fs::hard_link(from, to)?; // 目标已存在时失败 (AlreadyExists)
163     fs::remove_file(from)
164 }
165
166 #[cfg(windows)]
167 fn free_space(dir: &Path) -> Option<u64> {
168     use std::os::windows::ffi::OsStrExt;
169     use windows_sys::Win32::Storage::FileSystem::GetDiskFreeSpaceExW;
170     let wide: Vec<u16> = dir.as_os_str().encode_wide().chain(std::iter::once(0)).collect();
171     let mut free: u64 = 0;
172     // SAFETY: 路径以 0 结尾、调用期间有效；只请求“调用者可用字节数”，另外两个输出传空指针。
173     let ok = unsafe { GetDiskFreeSpaceExW(wide.as_ptr(), &mut free, std::ptr::null_mut(), std::ptr::null_mut()) };
174     (ok != 0).then_some(free)
175 }
176
177 #[cfg(not(windows))]
178 fn free_space(_dir: &Path) -> Option<u64> {
179     None
180 }
```



给 windows-sys 加一个模块

● ● ●

Cargo.toml

crates/lanshare/

第 1-20 行 • 共 20 行

```
1 [package]
2 name = "lanshare"
3 description = "局域网快传：服务端 + Windows 桌面外壳"
4 version.workspace = true
5 edition.workspace = true
6 publish.workspace = true
7
8 [[bin]]
9 name = "LanShare"
10 path = "src/main.rs"
11
12 [dependencies]
13 dirs = "6"
14 if-addr = "0.13"
15 lanshare-proto = { path = "../lanshare-proto" }
16 serde = { version = "1", features = ["derive"] }
17 windows-sys = { version = "0.61.2", features = ["Win32_System_SystemInformation", "Win32_Storage_FileSystem"] } 临时
18
19 [dev-dependencies]
20 tempfile = "3"
```



测试：六个场景

storage.rs crates/lanshare/src/

第 240–259 行 · 共 259 行

```

240         rename_no_replace(&from, &free).unwrap();
241         assert_eq!(fs::read(&free).unwrap(), b"new");
242         assert!(!from.exists());
243     }
244
245     #[test]
246     #[cfg(windows)]
247     fn reports_free_space() {
248         let (_d, s) = setup();
249         assert!(s.available_space().is_some_and(|n| n > 0));
250     }
251
252     #[test]
253     fn recreates_deleted_root() {
254         let (_d, s) = setup();
255         fs::remove_dir_all(s.root()).unwrap();
256         assert!(s.list().is_empty());
257         assert!(s.root().is_dir());
258     }
259 }

```



登记 storage

lib.rs

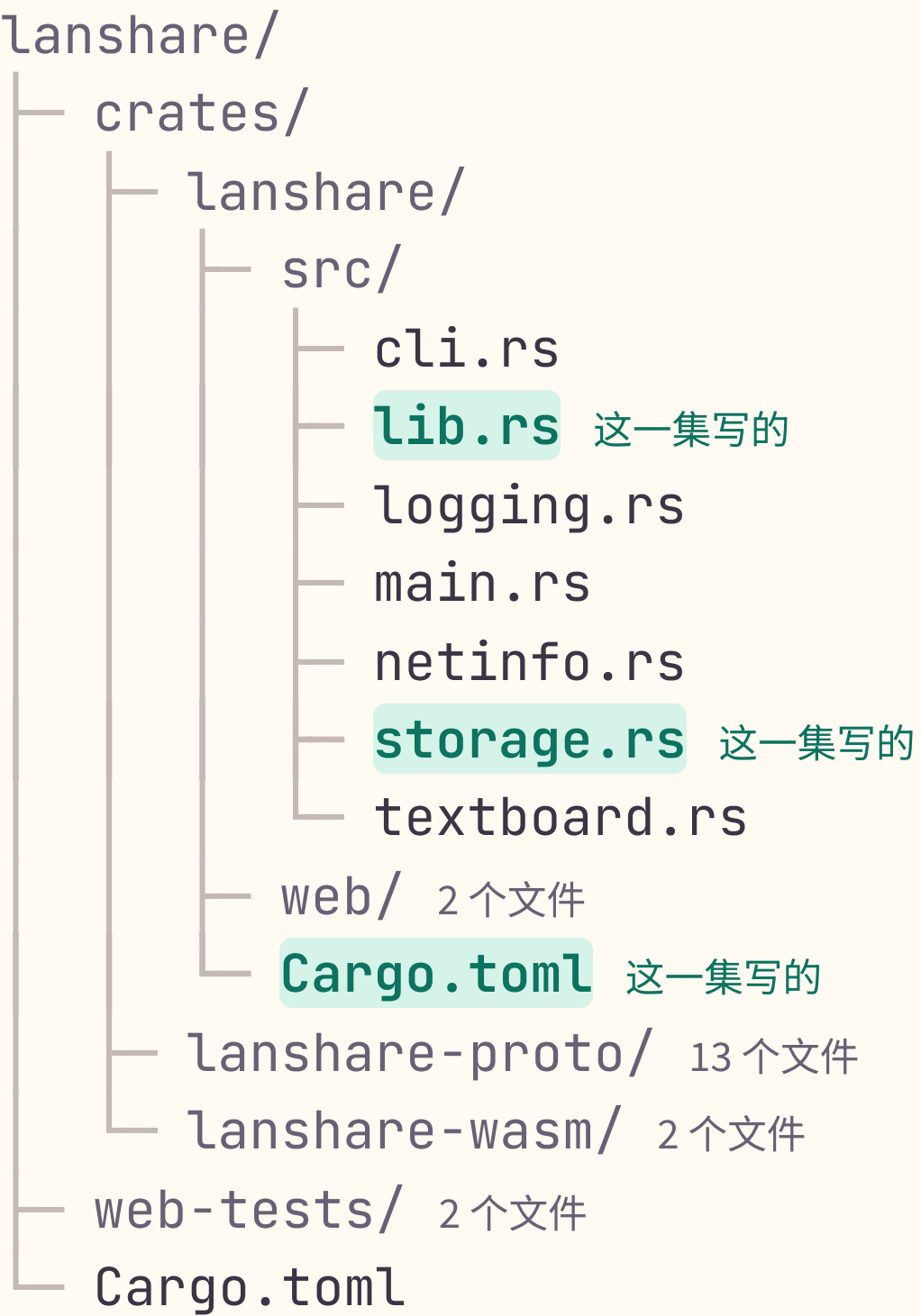
crates/lanshare/src/

第 1-7 行 · 共 7 行

```
1  //! 局域网快传 v2：服务端与桌面外壳的库部分（`main.rs` 只负责启动）。
2
3  pub mod cli;
4  pub mod logging;
5  pub mod netinfo;
6  pub mod storage;
7  pub mod textboard;
```



存储写好了





编译、跑测试

```
Windows PowerShell

PS D:\code\lanshare> cargo build
  Compiling lanshare-proto v2.0.0 (D:\code\lanshare\crates\lanshare-proto)
  Compiling lanshare v2.0.0 (D:\code\lanshare\crates\lanshare)
  Compiling lanshare-wasm v2.0.0 (D:\code\lanshare\crates\lanshare-wasm)
  Finished `dev` profile [unoptimized + debuginfo] target(s) in 3.14s

PS D:\code\lanshare> cargo test -p lanshare --lib
  Compiling lanshare-proto v2.0.0 (D:\code\lanshare\crates\lanshare-proto)
  Compiling lanshare v2.0.0 (D:\code\lanshare\crates\lanshare)
  Finished `test` profile [unoptimized + debuginfo] target(s) in 2.84s
  Running unittests src\lib.rs (target\debug\deps\lanshare-82b7b86216abeef3.exe)

running 22 tests
..... (省略 13 行, 都是上一集的测试, ok)
test storage::tests::reports_free_space ... ok
test cli::tests::default_dir_ends_with_lanshare ... ok
test storage::tests::recreates_deleted_root ... ok
test storage::tests::resolve_rejects_escapes ... ok
test storage::tests::list_hides_temp_hidden_and_dirs ... ok
test logging::tests::creates_missing_directory ... ok
test storage::tests::rename_never_overwrites ... ok
```



真实踩坑：变异测试第一次没抓住

● 修完之后，故意把 `rename_no_replace` 改回 `fs::rename`

这叫变异测试：把防线改坏，看测试会不会变红

● 结果：所有测试照样全绿

坏代码活了下来

● 原因：`exists()` 把问题掩盖了

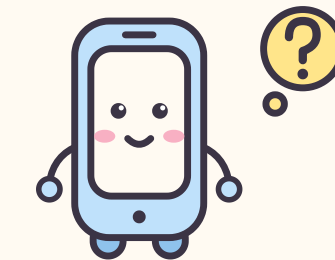
测试里没有“查完之后有人抢先建文件”的情况

● 去掉 `exists()` 之后，同样的变异立刻被测试抓住

测试全绿不等于防线有效，要把防线改坏一次试试

● 插曲：恢复了代码，测试却还是失败

脚本恢复的文件带着旧的修改时间，`cargo` 以为没改，测的仍是变异体



选择题

在 Windows 上，目标文件已经存在时，Rust 的 `fs::rename(a, b)` 会怎样？

A 返回 `AlreadyExists` 错误

B 成功返回，b 被覆盖

C 程序崩溃



选择题

在 Windows 上，目标文件已经存在时，Rust 的 `fs::rename(a, b)` 会怎样？

A 返回 `AlreadyExists` 错误

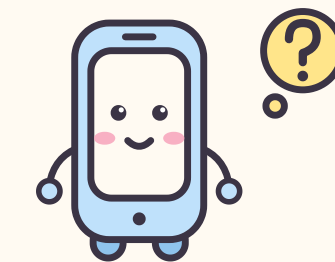
这是 Python 的 `os.rename` 在 Windows 上的行为，Rust 不是。

C 程序崩溃

不会崩溃，它“成功”了，这才可怕。

B 成功返回，b 被覆盖

Rust 的 `fs` 的 `rename` 会直接覆盖已存在的目标，而且返回成功。要不覆盖，得自己调用 `MoveFileExW`，不带替换标志。



选择题

下面这段代码有什么问题？

```
1 if !target.exists() {  
2     rename_no_replace(temp, &target)?;  
3 }
```

A 没问题，先查再改更保险

B 多此一举：rename_no_replace 自己就会在目标存在时失败，而且没有缝

C exists 会把文件删掉



选择题

下面这段代码有什么问题？

```
1 if !target.exists() {  
2     rename_no_replace(temp, &target)?;  
3 }
```

A 没问题，先查再改更保险

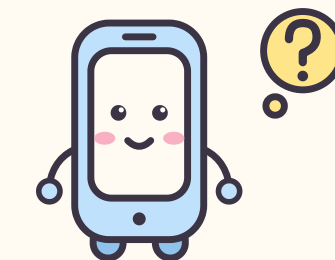
先查再改中间有一道缝，查完之后别的程序可能正好建了同名文件。

C exists 会把文件删掉

exists 只是查询，不改任何东西。

B 多此一举：rename_no_replace 自己就会在目标存在时失败，而且没有缝

不覆盖的改名把检查和改名合成了一步。多加一句 exists 不会更安全，反而会像那次变异测试一样，把问题掩盖起来。



选择题

candidate_names 去掉 move，会怎样？

```
1 std::iter::once(name.to_string()).chain((1u32..).map(|n| format!("{stem} ({n}){ext}")))
```

A 照常运行

B 编译报错：闭包借用的局部变量活得不够长

C 生成的名字全是空的



选择题

candidate_names 去掉 move，会怎样？

```
1 std::iter::once(name.to_string()).chain((1u32..).map(|n| format!("{stem} ({n}){ext}")))
```

A 照常运行

闭包会借用局部变量 stem 和 ext，而它们在函数返回时就消失了。

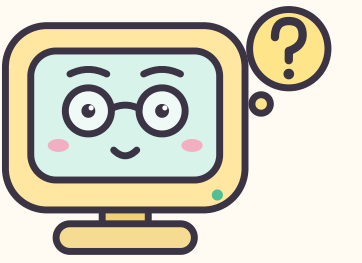
C 生成的名字全是空的

这种错误 Rust 在编译时就拦下了，运行不到。

B 编译报错：闭包借用的局部变量活得不够长

返回的迭代器比函数活得久，闭包却借着函数里的局部变量。加上 move，把这两个切片搬进闭包，问题就解决了。

想一想



finalize 已经用了不覆盖的改名，为什么还要 rename_lock 这把锁？去掉它，两个同名上传同时完成，会出什么事？



想一想

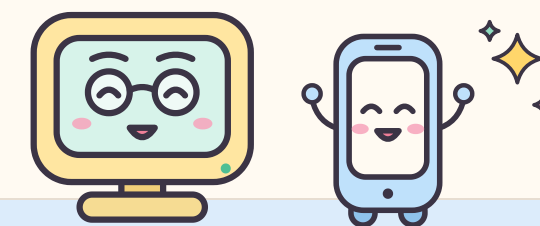
finalize 已经用了不覆盖的改名，为什么还要 rename_lock 这把锁？去掉它，两个同名上传同时完成，会出什么事？

不会覆盖，因为改名本身不覆盖，后到的会得到 AlreadyExists，换下一个名字。所以去掉锁，数据仍然安全。锁的作用是让同一个程序里的并发上传排队、按顺序选名字，少走冤枉路，结果也更可预测。两样各管一半：锁管我们自己的并发，不覆盖的改名管锁管不到的别的程序。



这一集写了什么

- ✓ 列文件和找文件：名字检查、canonicalize 后父目录必须是共享目录、只认普通文件
- ✓ 返回迭代器：impl Iterator 加 '_' 表示借用了参数；move 把切片搬进闭包
- ✓ Rust 的 fs::rename 在 Windows 上会覆盖，Python 的 os.rename 不会；同名函数，语义不同
- ✓ 不先查 exists()，直接用 MoveFileExW 做不覆盖的改名，被占了就换下一个名字
- ✓ 变异测试：把防线改坏一次，测试变红才说明它真的有效



下一集

全课完结，感谢收看